REAL TIME OPERATING SYSTEM

RTOS LAB

Assignment 2

Name: **Varun B Karmikanda**

USN: 261100610029

Program: ME Computer Science & Engineering

November 4, 2026

1. Priority Inversion

Question: Demonstrate the classic priority-inversion problem using three FreeRTOS tasks.

- **Task 1 – HIGH:** priority `tskIDLE_PRIORITY + 3`; blinks PA0 with 500 ms on/off intervals.
- **Task 2 – LOW:** priority `tskIDLE_PRIORITY + 1`; blinks PA1 with 500 ms on/off intervals and uses the shared binary semaphore.
- **Task 3 – MEDIUM:** priority `tskIDLE_PRIORITY + 2`; blinks PC13 and busy-waits for 500 ms for five cycles, then sleeps for 500 ms.

Code:

```
/*
 * priority-inversion/main.c
 *
 * Demonstrates the classic priority-inversion problem:
 *
 *   Task1  HIGH  (tskIDLE_PRIORITY + 3)  blinks PA0, 500 ms on/off
 *   Task2  LOW   (tskIDLE_PRIORITY + 1)  blinks PA1, 500 ms on/off
 *   Task3  MED   (tskIDLE_PRIORITY + 2)  blinks PC13, busy-wait 500 ms x 5
 *                                         cycles then sleeps 500 ms once
 *
 * PA0 and PA1 are collectively protected by a BINARY SEMAPHORE
 * (not a mutex).  PC13 is unprotected and Task3 never touches the
 * semaphore.
 *
 * Why a binary semaphore, not xSemaphoreCreateMutex()?
 * A FreeRTOS mutex has priority inheritance: the instant a high-priority
 * task blocks on it, the kernel temporarily raises the holder to the
 * high-priority task's level so it can run and release the mutex.
 * That's the FIX for priority inversion, not the bug.  A binary
 * semaphore has zero awareness of task priorities - it will never
 * bump anyone's priority.  Using it here lets the inversion play out
 * without any automatic mitigation.
 *
 * THE INVERSION SEQUENCE
 * ----------------------
 * T=0   Task2 (LOW) wakes, takes the semaphore, starts blinking PA1.
 *       It immediately vTaskDelay(500) - now it is Blocked.
 *
 * T~0   Task3 (MED) wakes, preempts Task2 (MED > LOW), and enters its
 *       busy-wait loop.  It spins for ~500 ms without ever calling
 *       vTaskDelay, so it stays in the Running/Ready state and the
 *       scheduler never picks Task2.
 *
 * T~500 Task1 (HIGH) wakes, tries to take the semaphore.
 *       Task2 still holds it (Task2 has been unable to run and
 *       release it because Task3 is ahead of it).
 *       Task1 blocks on the semaphore.
 *
 * Result: HIGH is blocked behind LOW, and LOW cannot run because
 *         MED is busy-waiting.  MED effectively blocks HIGH.
 *         This is priority inversion.
 *
 * Task3 loops 5 times (5 x 500 ms = ~5 s of busy-wait) then finally
 * calls vTaskDelay(500).  At that point Task2 is the highest Ready
 * task, runs, toggles PA1, then gives the semaphore.  Task1 unblocks
 * immediately - it has been waiting the entire 5 s.
 *
 * HARDWARE
 * --------
 * PA0  - LED1, active-high, external LED + ~330R to GND
 * PA1  - LED2, active-high, external LED + ~330R to GND
 * PC13 - onboard LED, active-low
 */
#include <stdint.h>
```

```c
#include "stm32f103_regs.h"
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"

/* ------------------------------------------------------------------ */
/* Priorities                                                          */
/* ------------------------------------------------------------------ */
#define PRIO_LOW    (tskIDLE_PRIORITY + 1)
#define PRIO_MED    (tskIDLE_PRIORITY + 2)
#define PRIO_HIGH   (tskIDLE_PRIORITY + 3)

/* ------------------------------------------------------------------ */
/* Shared resource protection - BINARY SEMAPHORE, not a mutex          */
/* A mutex would silently fix the inversion via priority inheritance.  */
/* ------------------------------------------------------------------ */
static SemaphoreHandle_t xGpioABinarySem;

/* ------------------------------------------------------------------ */
/* Busy-wait: spins on SysTick for the requested milliseconds.         */
/* Does NOT call vTaskDelay, so the calling task never leaves Running. */
/* This is the mechanism that lets Task3 starve Task2.                 */
/* ------------------------------------------------------------------ */
static volatile uint32_t g_ticks = 0;   /* incremented by tick hook */

void vApplicationTickHook(void)
{
    g_ticks++;
}

static void busy_wait_ms(uint32_t ms)
{
    uint32_t start = g_ticks;
    while ((g_ticks - start) < ms) {
        /* spin - deliberately consuming the CPU */
    }
}

/* ------------------------------------------------------------------ */
/* GPIO helpers                                                        */
/* ------------------------------------------------------------------ */
static void gpio_output_init(GPIO_TypeDef *port, uint32_t pin)
{
    volatile uint32_t *cr = (pin < 8) ? &port->CRL : &port->CRH;
    uint32_t shift = (pin % 8U) * 4U;
    *cr &= ~(0xFU << shift);
    *cr |=  (GPIO_CNF_MODE_OUTPUT_PP_2MHZ << shift);
}

/* ------------------------------------------------------------------ */
/* Task1 - HIGH priority - blinks PA0 - needs the semaphore            */
/* ------------------------------------------------------------------ */
static void task1_high(void *pvParameters)
{
    (void)pvParameters;
    /* give Task2 a head-start so it can acquire the semaphore first */
    vTaskDelay(pdMS_TO_TICKS(10));

    for (;;) {
        /* --- WAIT for the shared GPIO-A semaphore --- */
        xSemaphoreTake(xGpioABinarySem, portMAX_DELAY);

        /* PA0 on */
        GPIOA->ODR |=  (1U << 0);
        vTaskDelay(pdMS_TO_TICKS(500));

        /* PA0 off */
        GPIOA->ODR &= ~(1U << 0);
        vTaskDelay(pdMS_TO_TICKS(500));

        xSemaphoreGive(xGpioABinarySem);
    }
}
```

```
128
129 /* ------------------------------------------------------------------ */
130 /* Task2 - LOW priority - blinks PA1 - needs the semaphore             */
131 /* ------------------------------------------------------------------ */
132 static void task2_low(void *pvParameters)
133 {
134     (void)pvParameters;
135     for (;;) {
136         /* Task2 acquires the semaphore first (it starts before Task1
137          * and Task3 because of the 10 ms head-start above).
138          * Once Task3 wakes and busy-waits, Task2 cannot run to give
139          * the semaphore back - trapping Task1. */
140         xSemaphoreTake(xGpioABinarySem, portMAX_DELAY);
141
142         /* PA1 on */
143         GPIOA->ODR |=  (1U << 1);
144         vTaskDelay(pdMS_TO_TICKS(500));
145
146         /* PA1 off */
147         GPIOA->ODR &= ~(1U << 1);
148         vTaskDelay(pdMS_TO_TICKS(500));
149
150         xSemaphoreGive(xGpioABinarySem);
151     }
152 }
153
154 /* ------------------------------------------------------------------ */
155 /* Task3 - MED priority - blinks PC13 - no semaphore                  */
156 /* Busy-waits 500 ms x 5 cycles (5 s total) then sleeps 500 ms once.  */
157 /* The busy-wait is what blocks Task2 from running and releasing the  */
158 /* semaphore that Task1 is waiting on.                                */
159 /* ------------------------------------------------------------------ */
160 static void task3_med(void *pvParameters)
161 {
162     (void)pvParameters;
163     for (;;) {
164         /* --- 5 busy-wait cycles of 500 ms each -------------------- */
165         for (uint32_t i = 0; i < 5; i++) {
166             /* PC13 toggle (active-low, so XOR gives a visible blink) */
167             GPIOC->ODR ^= (1U << 13);
168
169             /* BUSY-WAIT: does NOT yield the CPU - this is the key.
170              * While spinning here, Task2 (which holds the semaphore)
171              * cannot run because Task3 (MED) outranks Task2 (LOW).
172              * Task1 (HIGH) is blocked on the semaphore.
173              * => MED is blocking HIGH through LOW: inversion. */
174             busy_wait_ms(500);
175         }
176
177         /* --- one cooperative sleep to eventually let things flow --- */
178         vTaskDelay(pdMS_TO_TICKS(500));
179     }
180 }
181
182 /* ------------------------------------------------------------------ */
183 /* main                                                               */
184 /* ------------------------------------------------------------------ */
185 int main(void)
186 {
187     /* enable clocks for GPIOA and GPIOC */
188     RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_IOPCEN;
189
190     gpio_output_init(GPIOA, 0);   /* PA0 - Task1 LED */
191     gpio_output_init(GPIOA, 1);   /* PA1 - Task2 LED */
192     gpio_output_init(GPIOC, 13);  /* PC13 - Task3 onboard LED */
193
194     /* BINARY semaphore - deliberately not a mutex */
195     xGpioABinarySem = xSemaphoreCreateBinary();
196     xSemaphoreGive(xGpioABinarySem);   /* start it available */
197
198     xTaskCreate(task1_high, "T1_HIGH", configMINIMAL_STACK_SIZE, NULL, PRIO_HIGH, NULL);
199     xTaskCreate(task2_low,  "T2_LOW",  configMINIMAL_STACK_SIZE, NULL, PRIO_LOW,  NULL);
200     xTaskCreate(task3_med,  "T3_MED",  configMINIMAL_STACK_SIZE, NULL, PRIO_MED,  NULL);
```

```
201
202     vTaskStartScheduler();
203
204     for (;;) {}
205 }
206
207 void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName)
208 {
209     (void)xTask; (void)pcTaskName;
210     for (;;) {}
211 }
```

Output and observation: Task 2 acquires the binary semaphore and then becomes delayed during its blink. Task 3 has a higher priority than Task 2 and busy-waits without calling `vTaskDelay()`, so Task 2 cannot run to release the semaphore. When Task 1 attempts to acquire the semaphore, it blocks behind the low priority Task 2. Therefore, the medium-priority task indirectly prevents the high-priority task from progressing. This is priority inversion.

Result: A binary semaphore does not provide priority inheritance, so the priority-inversion behavior is observed.

2. Shared LED Without Synchronization

Question: Run two same-priority tasks that toggle the same GPIO pin without a mutex, critical section, or other coordination.

- `task1_toggle_1hz`: toggles PC13, delays 1000 ms, toggles again, and delays 1000 ms.
- `task2_toggle_2hz`: toggles PC13, delays 500 ms, toggles again, and delays 500 ms.

Code:

```
/*
 * shared-led-no-sync/main.c
 *
 * Two tasks, same priority, both toggling the SAME GPIO pin (PC13, the
 * Bluepill's onboard LED) with no mutex, no critical section, no
 * coordination between them at all. Each task's loop is now explicit
 * about on/off rather than a single toggle per iteration:
 *
 *   task1_toggle_1hz - on, delay 1000 ms, off, delay 1000 ms
 *                      -> one complete blink every 2 s
 *   task2_toggle_2hz - on, delay  500 ms, off, delay  500 ms
 *                      -> one complete blink every 1 s
 *
 * The underlying toggle rate on the pin is unchanged from the simpler
 * version of this file - this only makes "on" and "off" explicit
 * instead of letting them fall out of alternating toggles.
 *
 * This is deliberately the "wrong" way to share a pin - see the note
 * at the bottom of this file for what that actually costs you.
 */
#include <stdint.h>
#include "stm32f103_regs.h"

#include "FreeRTOS.h"
#include "task.h"

static void gpio_output_init(GPIO_TypeDef *port, uint32_t pin)
{
    volatile uint32_t *cr = (pin < 8) ? &port->CRL : &port->CRH;
    uint32_t shift = (pin % 8U) * 4U;

    *cr &= ~(0xFU << shift);
    *cr |=  (GPIO_CNF_MODE_OUTPUT_PP_2MHZ << shift);
}

static void task1_toggle_1hz(void *pvParameters)
{
    (void)pvParameters;
    for (;;) {
        GPIOC->ODR ^= (1U << 13);           /* on */
        vTaskDelay(pdMS_TO_TICKS(1000));
        GPIOC->ODR ^= (1U << 13);           /* off */
        vTaskDelay(pdMS_TO_TICKS(1000));
        /* one full loop iteration = one complete blink, 2000 ms total */
    }
}

static void task2_toggle_2hz(void *pvParameters)
{
    (void)pvParameters;
    for (;;) {
        GPIOC->ODR ^= (1U << 13);           /* on */
        vTaskDelay(pdMS_TO_TICKS(500));
        GPIOC->ODR ^= (1U << 13);           /* off */
        vTaskDelay(pdMS_TO_TICKS(500));
        /* one full loop iteration = one complete blink, 1000 ms total */
    }
```

```
}

int main(void)
{
    RCC->APB2ENR |= RCC_APB2ENR_IOPCEN;
    gpio_output_init(GPIOC, 13);

    xTaskCreate(task1_toggle_1hz, "task1", configMINIMAL_STACK_SIZE, NULL,
        tskIDLE_PRIORITY + 1, NULL);
    xTaskCreate(task2_toggle_2hz, "task2", configMINIMAL_STACK_SIZE, NULL,
        tskIDLE_PRIORITY + 1, NULL);

    vTaskStartScheduler();

    for (;;) {
        /* only reached if vTaskStartScheduler() fails, e.g. out of heap */
    }
}

/* Required because FreeRTOSConfig.h sets configCHECK_FOR_STACK_OVERFLOW=2 */
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName)
{
    (void)xTask;
    (void)pcTaskName;
    for (;;) {
    }
}

/*
 * What "no synchronization" actually costs here:
 *
 * GPIOC->ODR ^= (1U << 13) is NOT a single atomic operation - it compiles
 * to a load of ODR, an XOR, and a store back to ODR. Both tasks run this
 * exact sequence on the exact same register.
 *
 * Since these two tasks share one priority, a SysTick-driven context
 * switch can land between the load and the store of either task's
 * toggle. If task1 is preempted right there, and task2 runs an entire
 * toggle to completion before task1 resumes, task1 finishes its own
 * store using the STALE value it loaded before being preempted -
 * silently undoing task2's toggle. One of the two intended edges on
 * PC13 is lost, with no error, no warning, and no way to tell from
 * the code alone that it happened.
 *
 * The intended behaviour visually - if nothing ever raced - would be
 * an LED that spends noticeably more time in one state than the other
 * (task2's extra mid-second toggle keeps flipping it). What you'll
 * actually observe, run for long enough, is that pattern occasionally
 * glitching - a "toggle" that doesn't visibly happen - as this race
 * resolves the wrong way. It's rare, not visible on every run, and
 * exactly the kind of bug that a mutex around this read-modify-write
 * (or simply using GPIOC->BSRR to set/reset the specific bit atomically
 * from a single write) makes impossible instead of merely unlikely.
 */
```

Output and observation: Both tasks access PC13 independently. The statement `GPIOC->ODR ^= (1U << 13)` is a read-modify-write operation rather than one atomic GPIO operation. A context switch between the read and write can cause one task to store an old value and silently cancel the other task's toggle. The LED may therefore show irregular or missing transitions.

Result: The shared LED is accessed without synchronization, producing a race condition and nondeterministic LED behavior.

3. Shared LED With Mutex

Question: Protect the complete blink operation of the two shared-LED tasks using a mutex.

Code:

```
/*
 * shared-led-mutex/main.c
 *
 * Same two tasks as shared-led-no-sync, but this time a mutex is held
 * across the ENTIRE blink - not just each individual toggle:
 *
 *   task1_toggle_1hz - take mutex; on, delay 1000, off, delay 1000; give mutex
 *   task2_toggle_2hz - take mutex; on, delay  500, off, delay  500; give mutex
 *
 * This does fully remove the lost-toggle race from Q V1/V2 - while one
 * task holds the mutex, the other cannot touch PC13 at all, not even
 * once. But see the note at the bottom of this file for what that
 * actually costs: it does not merely fix the race, it changes the
 * program's behaviour altogether.
 */
#include <stdint.h>
#include "stm32f103_regs.h"

#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"

static SemaphoreHandle_t xLedMutex;

static void gpio_output_init(GPIO_TypeDef *port, uint32_t pin)
{
    volatile uint32_t *cr = (pin < 8) ? &port->CRL : &port->CRH;
    uint32_t shift = (pin % 8U) * 4U;

    *cr &= ~(0xFU << shift);
    *cr |=  (GPIO_CNF_MODE_OUTPUT_PP_2MHZ << shift);
}

static void task1_toggle_1hz(void *pvParameters)
{
    (void)pvParameters;
    for (;;) {
        xSemaphoreTake(xLedMutex, portMAX_DELAY);
        GPIOC->ODR ^= (1U << 13);           /* on */
        vTaskDelay(pdMS_TO_TICKS(1000));
        GPIOC->ODR ^= (1U << 13);           /* off */
        vTaskDelay(pdMS_TO_TICKS(1000));
        xSemaphoreGive(xLedMutex);
        /* the mutex is held for the WHOLE 2 s blink, not just each toggle */
    }
}

static void task2_toggle_2hz(void *pvParameters)
{
    (void)pvParameters;
    for (;;) {
        xSemaphoreTake(xLedMutex, portMAX_DELAY);
        GPIOC->ODR ^= (1U << 13);           /* on */
        vTaskDelay(pdMS_TO_TICKS(500));
        GPIOC->ODR ^= (1U << 13);           /* off */
        vTaskDelay(pdMS_TO_TICKS(500));
        xSemaphoreGive(xLedMutex);
        /* held for the WHOLE 1 s blink, not just each toggle */
    }
}

int main(void)
{
    RCC->APB2ENR |= RCC_APB2ENR_IOPCEN;
    gpio_output_init(GPIOC, 13);

```

```
67      xLedMutex = xSemaphoreCreateMutex();
68
69      xTaskCreate(task1_toggle_1hz, "task1", configMINIMAL_STACK_SIZE, NULL,
            tskIDLE_PRIORITY + 1, NULL);
70      xTaskCreate(task2_toggle_2hz, "task2", configMINIMAL_STACK_SIZE, NULL,
            tskIDLE_PRIORITY + 1, NULL);
71
72      vTaskStartScheduler();
73
74      for (;;) {
75          /* only reached if vTaskStartScheduler() fails, e.g. out of heap */
76      }
77  }
78
79  /* Required because FreeRTOSConfig.h sets configCHECK_FOR_STACK_OVERFLOW=2 */
80  void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName)
81  {
82      (void)xTask;
83      (void)pcTaskName;
84      for (;;) {
85      }
86  }
87
88  /*
89   * What synchronizing the WHOLE blink actually does to the behaviour:
90   *
91   * The lost-toggle race from Q V1/V2 is completely gone - correct, and
92   * exactly what a mutex is for. But holding the mutex across the full
93   * on-delay-off-delay sequence means each task now blocks the OTHER
94   * one out of running its own blink at all, for as long as the first
95   * one's entire cycle takes:
96   *
97   *    - Whichever task acquires the mutex first runs its full blink
98   *      uninterrupted - task1's full 2 s cycle, or task2's full 1 s
99   *      cycle - while the other task sits blocked on xSemaphoreTake(),
100  *      unable to touch the pin even though it isn't actually racing
101  *      with anything during that time.
102  *    - If task1 wins the mutex, task2's intended "twice a second"
103  *      schedule is not just delayed slightly - it is blocked for up to
104  *      2 full seconds, then it runs its own complete 1 s blink, then
105  *      task1 (which has been waiting the whole time) gets its turn.
106  *    - The two tasks end up taking turns running whole blinks back to
107  *      back, rather than interleaving on their own independent
108  *      schedules. Neither task's 1 Hz / 2 Hz timing survives contact
109  *      with the other once the WHOLE operation is what's serialized.
110  *
111  * This is the real trade-off "lock the whole operation" always makes:
112  * it buys correctness on the shared resource by giving up the
113  * independence the two tasks were supposed to have. A narrower fix -
114  * a mutex (or just GPIOC->BSRR) around only the individual toggle,
115  * not the delays around it - would remove the race while leaving each
116  * task free to keep its own timing. Whether that trade-off is
117  * acceptable depends entirely on whether "two independently-timed
118  * blinkers" or "one shared correct sequence" is actually the goal.
119  */
```

Output and observation: Only the task holding `xLedMutex` can access PC13. The mutex is held across the complete on-delay-off-delay sequence. Consequently, the lost-toggle race is removed. However, the other task must wait for the complete blink cycle, so the two independent blink schedules no longer interleave.

Result: The mutex prevents simultaneous access to the shared LED and eliminates the race condition. Holding the mutex over the whole blink gives correct mutual exclusion but reduces task independence.

4. Deadlock and Synchronization

Question: Create a deadlock and solve the deadlock using the waiting method and the binary mutex method.

4.1 Deadlock condition

Two tasks use two resources, `mutex_A` and `mutex_B`, in opposite order:

- Task 1 takes A and waits for B.
- Task 2 takes B and waits for A.

Neither task can continue, because each task is waiting for a resource held by the other. This is a circular wait and produces deadlock.

Code used for the experiment:

```
#include <stdint.h>
#include "stm32f103_regs.h"
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"

SemaphoreHandle_t mutex_A;
SemaphoreHandle_t mutex_B;

typedef struct {
    GPIO_TypeDef    *port;
    uint32_t        pin;
    uint32_t        period_ms;
} led_task_args_t;

static const led_task_args_t led_a = { GPIOA, 0, 500 };
static const led_task_args_t led_b = { GPIOA, 1, 500 };

static void gpio_output_init(GPIO_TypeDef *port, uint32_t pin){
    volatile uint32_t *cr = (pin < 8) ? &port->CRL : &port->CRH;
    uint32_t shift = (pin % 8U) * 4U;

    *cr &= ~(0xFU << shift);
    *cr |= (GPIO_CNF_MODE_OUTPUT_PP_2MHZ << shift);
}

static void led_task_1(void *pvParameters){
    const led_task_args_t *args = (const led_task_args_t *)pvParameters;

    for (;;)
    {
        xSemaphoreTake(mutex_A, portMAX_DELAY);

        if(xSemaphoreTake(mutex_B, pdMS_TO_TICKS(500)) == pdPASS){
            args->port->ODR ^= (1U << args->pin);
            vTaskDelay(pdMS_TO_TICKS(args->period_ms));

            args->port->ODR ^= (1U << args->pin);
            vTaskDelay(pdMS_TO_TICKS(args->period_ms));

            xSemaphoreGive(mutex_B);
        }


        xSemaphoreGive(mutex_A);

        vTaskDelay(pdMS_TO_TICKS(100));

    }
}

static void led_task_2(void *pvParameters){
```

```
53      const led_task_args_t *args = (const led_task_args_t *)pvParameters;
54
55      for (;;)
56      {
57          xSemaphoreTake(mutex_B, portMAX_DELAY);
58
59          if(xSemaphoreTake(mutex_A, pdMS_TO_TICKS(500)) == pdPASS){
60              args->port->ODR ^= (1U << args->pin);
61              vTaskDelay(pdMS_TO_TICKS(args->period_ms));
62
63              args->port->ODR ^= (1U << args->pin);
64              vTaskDelay(pdMS_TO_TICKS(args->period_ms));
65
66              xSemaphoreGive(mutex_A);
67          }
68
69
70          xSemaphoreGive(mutex_B);
71
72          vTaskDelay(pdMS_TO_TICKS(100));
73
74      }
75  }
76
77  int main(void){
78
79      mutex_A = xSemaphoreCreateMutex();
80      mutex_B = xSemaphoreCreateMutex();
81
82      if(mutex_A == NULL || mutex_B == NULL){
83          for(;;) {}
84      }
85
86      RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;
87
88      gpio_output_init(led_a.port, led_a.pin);
89      gpio_output_init(led_b.port, led_b.pin);
90
91      xTaskCreate(led_task_1, "thread_1", configMINIMAL_STACK_SIZE, (void *)&led_a,
            tskIDLE_PRIORITY + 1, NULL);
92      xTaskCreate(led_task_2, "thread_2", configMINIMAL_STACK_SIZE, (void *)&led_b,
            tskIDLE_PRIORITY + 1, NULL);
93
94      vTaskStartScheduler();
95
96      for(;;){}
97  }
98
99  void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName)
100 {
101     (void)xTask;
102     (void)pcTaskName;
103     for (;;) {}
104 }
```

4.2 Waiting method

The program uses a bounded wait when taking the second mutex:

```
1  if (xSemaphoreTake(mutex_B, pdMS_TO_TICKS(500)) == pdPASS) {
2      /* use the second resource */
3      xSemaphoreGive(mutex_B);
4  }
5  xSemaphoreGive(mutex_A);
```

If the second resource is unavailable for 500 ms, the task gives back the first resource and retries later. This breaks the circular wait and prevents permanent deadlock.

4.3 Binary mutex method

The deadlock can also be solved by protecting the complete operation with one shared mutex instead of taking two resources in opposite order:

```
SemaphoreHandle_t shared_mutex;

void task(void *argument)
{
    for (;;) {
        xSemaphoreTake(shared_mutex, portMAX_DELAY);
        /* access both resources in one protected section */
        use_resource_A();
        use_resource_B();
        xSemaphoreGive(shared_mutex);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

Only one task can enter the protected section at a time, so a task cannot hold one resource while waiting for the other. The circular-wait condition is removed.

4.4 Output and result

With the bounded waiting method, a task that cannot obtain the second mutex times out, releases its first mutex, and retries. With the single binary mutex method, access is serialized from the beginning. Both methods prevent the system from remaining permanently deadlocked.